

Introducción a Ionic

Fundamentos técnicos y contexto en el desarrollo moderno

1. ¿Qué es Ionic?

Ionic es un **framework de desarrollo de aplicaciones híbridas** que permite crear aplicaciones móviles, web y de escritorio utilizando tecnologías web estándar:

- HTML5
- CSS3
- JavaScript / TypeScript

Se apoya en frameworks modernos como:

- Angular
- React
- Vue

y utiliza **Capacitor** (o anteriormente Cordova) para acceder a funcionalidades nativas del dispositivo.

2. Arquitectura de Ionic

A nivel técnico, Ionic se estructura en varias capas:

2.1 Capa de presentación

- Conjunto de **componentes UI reutilizables**
- Basados en Web Components
- Renderizados mediante Shadow DOM

Ejemplo:

```
<ion-button expand="full">  
  Guardar  
</ion-button>
```

2.2 Capa lógica

- Implementada en Angular / React / Vue
- Manejo de estado
- Control de eventos
- Comunicación con servicios

2.3 Capa nativa (Capacitor)

Permite acceder a APIs del sistema operativo:

- Cámara
- Geolocalización
- Sistema de archivos
- Notificaciones

3. ¿Por qué se utiliza Ionic?

3.1 Reutilización de código

Un único código base permite generar:

- Aplicaciones Android
- Aplicaciones iOS
- Aplicaciones Web (PWA)

3.2 Productividad

- No requiere conocimientos profundos de desarrollo nativo
- Permite trabajar con tecnologías web conocidas

3.3 Ecosistema JavaScript

Se integra perfectamente con:

- Node.js (backend)
- APIs REST
- Librerías modernas

4. Conceptos técnicos fundamentales

4.1 Web Components

Ionic basa su UI en Web Components, lo que implica:

- Encapsulación del DOM
- Reutilización de componentes
- Independencia del framework

Ejemplo conceptual:

```
class MiComponente extends HTMLElement {  
  constructor() {  
    super();  
  }  
}
```

4.2 Shadow DOM

Permite encapsular estilos y estructura:

- Evita conflictos CSS
- Aísla componentes

4.3 Virtual DOM (dependiendo del framework)

Si se usa:

- React → Virtual DOM
- Angular → Change detection

4.4 Routing

Ionic implementa navegación tipo SPA:

```
const routes = [  
  { path: '', component: HomePage },  
  { path: 'detalle', component: DetailPage }  
];
```

4.5 Observables y asincronía

En Angular/Ionic:

- Uso intensivo de **Observables** (RxJS)
- Integración con **async/await**

Ejemplo:

```
async cargarDatos() {  
  const data = await this.apiService.getDatos();  
}
```

4.6 Consumo de APIs (conexión con Node.js)

Ionic consume APIs REST como las que habéis construido:

```
this.http.get("http://localhost:3000/api/users")
```

Conceptos relacionados:

- JSON
- HTTP
- Promesas
- Event Loop

4.7 Gestión de estado

Dependiendo del framework:

- Angular → servicios + RxJS
- React → hooks (**useState**, **useEffect**)
- Vue → reactive state

4.8 CLI (Command Line Interface)

Herramienta fundamental:

ionic start myApp

ionic serve

ionic build

Permite:

- Crear proyectos
- Generar componentes
- Compilar apps

5. Alternativas a Ionic

5.1 React Native

- Usa JavaScript
- Renderiza componentes nativos reales
- Mejor rendimiento
- Mayor complejidad

5.2 Flutter

- Lenguaje Dart
- Renderizado propio
- Alta performance
- Curva de aprendizaje mayor

5.3 NativeScript

- Acceso directo a APIs nativas
- Menor adopción

5.4 Desarrollo nativo

- Java/Kotlin (Android)
- Swift (iOS)
- Máximo rendimiento
- Mayor coste de desarrollo

Comparativa técnica

Tecnología	Lenguaje	UI	Rendimiento	Complejidad
Ionic	JS / TS	Web UI	Medio	Baja
React Native	JS / TS	Nativo	Alto	Media
Flutter	Dart	Propio	Muy alto	Alta
Nativo	Java/Swift	Nativo	Muy alto	Muy alta

6. Flujo de ejecución en Ionic

1. Usuario interactúa con UI (`ion-button`)
2. Evento se captura en componente
3. Se ejecuta lógica JS (`async/await`)
4. Se llama a API (`Node.js`)
5. Se actualiza estado y vista

7. Integración con JavaScript moderno

Variables modernas

```
const usuarios = await this.service.getUsuarios();
```

Arrow functions

```
usuarios.map(u => u.nombre);
```

Destructuring

```
const { nombre, email } = usuario;
```

Módulos

```
import { ApiService } from './services/api.service';
```

Programación funcional

```
const activos = usuarios.filter(u => u.activo);
```

8. Ventajas técnicas

- Basado en estándares web
- Gran comunidad
- Compatible con PWA
- Fácil integración con backend Node.js
- Excelente para aplicaciones empresariales

9. Limitaciones

- Rendimiento inferior a nativo en apps complejas
- Dependencia del WebView
- Limitaciones en animaciones avanzadas
- Acceso a hardware depende de plugins

Primeros pasos con Ionic

Requisitos y configuración del entorno

1. Requisitos previos

Para trabajar correctamente con Ionic es imprescindible cumplir una serie de requisitos técnicos. Ionic no funciona de forma aislada, sino que depende del ecosistema de Node.js.

1.1 Node.js

Ionic se ejecuta sobre Node.js, por lo que es el requisito principal.

Instalación

Recomendado: versión LTS

```
node -v
```

```
npm -v
```

Consideraciones técnicas

- Node gestiona dependencias y ejecución
- npm permite instalar Ionic CLI y librerías
- Todo el proceso de build se basa en Node

1.2 npm (Node Package Manager)

Incluido con Node.js.

Responsabilidades:

- Gestión de dependencias
- Scripts de ejecución
- Resolución de paquetes

Ejemplo de uso:

```
npm install
```

```
npm run start
```

1.3 Conocimientos previos imprescindibles

Antes de trabajar con Ionic, el alumno debe dominar:

- ES6+
- Async/await
- Promesas
- Módulos (`import/export`)
- Programación funcional básica
- Consumo de APIs REST

Esto es clave porque Ionic no abstrae JavaScript, lo utiliza plenamente.

2. Instalación de Ionic CLI

El CLI es la herramienta principal de trabajo.

```
npm install -g @ionic/cli
```

Verificación:

```
ionic -v
```

Funciones del CLI

- Crear proyectos
- Ejecutar servidor de desarrollo
- Generar componentes/páginas
- Compilar aplicaciones
- Integración con Capacitor

3. Creación de un proyecto

Comando básico

`ionic start myApp blank`

Selección de framework

Durante la creación:

- Angular
- React
- Vue

Ejemplo:

`ionic start myApp blank --type=angular`

Estructura generada

```
/myApp
├── src/
│   ├── app/
│   ├── assets/
│   ├── index.html
│   └── main.ts
├── angular.json
├── package.json
└── ionic.config.json
```

4. Puesta en marcha del proyecto

Ejecutar aplicación

```
cd myApp  
ionic serve
```

Qué sucede internamente

- Se lanza un servidor de desarrollo
- Se compila el proyecto (Angular/React/Vue)
- Se abre en navegador (WebView)

5. Uso de Capacitor (acceso nativo)

Ionic utiliza Capacitor para interactuar con el sistema operativo.

Inicialización

```
ionic build  
npx cap init
```

Añadir plataforma

```
npx cap add android  
npx cap add ios
```

Sincronización

```
npx cap sync
```

6. Entorno de desarrollo recomendado

VS Code

Extensiones clave:

- Angular Language Service
- Ionic Snippets
- ESLint
- Prettier

Navegador

- Chrome (recomendado)
- Uso intensivo de DevTools

7. Flujo de trabajo básico

El ciclo habitual de desarrollo es:

1. Crear proyecto
2. Ejecutar en navegador
3. Modificar componentes
4. Consumir APIs
5. Testear
6. Compilar para dispositivo

8. Ejemplo básico de integración con JavaScript moderno

Archivo de componente:

```
async cargarDatos() {
  try {
    const response = await fetch("https://dummyjson.com/products");
    const data = await response.json();

    this.products = data.products.filter(p => p.price > 50);

  } catch (error) {
    console.error(error);
  }
}
```

Conceptos aplicados:

- async/await
- fetch
- filter
- manejo de errores

9. Gestión de dependencias

El archivo `package.json` contiene:

- Dependencias del proyecto
- Scripts
- Configuración

Ejemplo:

```
{
  "dependencies": {
    "@ionic/angular": "^6.0.0",
    "rxjs": "^7.0.0"
  }
}
```

10. Construcción (build)

ionic build

Genera:

/www

Que contiene la app compilada.

11. Ejecución en dispositivo

Android

npx cap open android

iOS (solo en macOS)

npx cap open ios

Requiere Xcode instalado.

12. Problemas comunes

Versiones incompatibles

- Node muy antiguo
- Ionic CLI desactualizado

Dependencias rotas

rm -rf node_modules

npm install

Problemas con rutas

Siempre trabajar desde el root del proyecto.

13. Buenas prácticas iniciales

- No modificar directamente archivos generados sin entender
- Separar lógica en servicios
- Usar async/await siempre que sea posible
- Evitar lógica pesada en componentes
- Aprovechar modularidad

14. Relación con Node.js y backend

Ionic no gestiona datos directamente, sino que:

- Consume APIs REST
- Utiliza HTTP
- Trabaja con JSON

Ejemplo:

```
this.http.get("http://localhost:300
```

Conclusión

Los primeros pasos en Ionic no consisten en aprender componentes visuales, sino en:

- Configurar correctamente el entorno
- Entender el flujo de trabajo
- Aplicar conocimientos de JavaScript moderno
- Integrar con servicios backend

Ionic es una extensión natural del ecosistema que ya se ha trabajado en Node.js.

Proyecto Ionic equivalente a API REST

Objetivo

Construir una app Ionic que:

- Consuma datos (simulando una API o real)
- Aplique arquitectura por capas:
 - **Pages (equivalente a routes)**
 - **Services (equivalente a services backend)**
 - **Controllers (integrados en componentes)**

1. Creación del proyecto

```
ionic start usersApp blank --type=angular
cd usersApp
ionic serve
```

2. Estructura del proyecto (adaptada)

Vamos a organizar el proyecto de forma profesional:

```
/src/app
├── pages/
│   └── users/
│       ├── users.page.ts
│       ├── users.page.html
│       └── users.page.scss
├── services/
│   └── users.service.ts
├── models/
│   └── user.model.ts
```

3. Modelo (equivalente a entidad backend)

Archivo: `models/user.model.ts`

```
export interface User {  
  id: number;  
  name: string;  
  email: string;  
  active: boolean;  
}
```

4. Service (equivalente a capa de negocio)

Archivo: `services/users.service.ts`

Este servicio simula una API REST usando async/await.

```
import { Injectable } from '@angular/core';
import { User } from '../models/user.model';

@Injectable({
  providedIn: 'root'
})
export class UsersService {

  private users: User[] = [
    { id: 1, name: 'Ana', email: 'ana@test.com', active: true },
    { id: 2, name: 'Luis', email: 'luis@test.com', active: false },
    { id: 3, name: 'Carlos', email: 'carlos@test.com', active: true }
  ];

  async getUsers(): Promise<User[]> {
    return new Promise(resolve => {
      setTimeout(() => resolve(this.users), 500);
    });
  }

  async getActiveUsers(): Promise<User[]> {
    const users = await this.getUsers();
    return users.filter(u => u.active);
  }

  async getUserById(id: number): Promise<User | undefined> {
    const users = await this.getUsers();
    return users.find(u => u.id === id);
  }
}
```

5. Page (equivalente a controller + route)

Archivo: `pages/users/users.page.ts`

Aquí se gestiona la lógica de presentación (equivalente al controller en backend).

```
import { Component, OnInit } from '@angular/core';
import { UsersService } from '../../services/users.service';
import { User } from '../../models/user.model';
```

```
@Component({
  selector: 'app-users',
  templateUrl: './users.page.html',
  styleUrls: ['./users.page.scss'],
})
export class UsersPage implements OnInit {

  users: User[] = [];
  loading = false;

  constructor(private usersService: UsersService) {}

  async ngOnInit() {
    await this.loadUsers();
  }

  async loadUsers() {
    try {
      this.loading = true;
      this.users = await this.usersService.getActiveUsers();
    } catch (error) {
      console.error('Error:', error);
    } finally {
      this.loading = false;
    }
  }
}
```

6. Vista (HTML)

Archivo: `users.page.html`

```
<ion-header>
  <ion-toolbar>
    <ion-title>Usuarios Activos</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content>

  <ion-spinner *ngIf="loading"></ion-spinner>

  <ion-list *ngIf="!loading">
    <ion-item *ngFor="let user of users">
      <ion-label>
        <h2>{{ user.name }}</h2>
        <p>{{ user.email }}</p>
      </ion-label>
    </ion-item>
  </ion-list>

</ion-content>
```

7. Routing (equivalente a routes backend)

Archivo: `app-routing.module.ts`

```
const routes: Routes = [  
  {  
    path: 'users',  
    loadChildren: () => import('./pages/users/users.module')  
      .then(m => m.UsersPageModule)  
  }  
];
```

8. Equivalencia con Node.js

Node.js	Ionic
Routes	Routing (Angular)
Controllers	Pages (component logic)
Services	Services
Models	Interfaces
Middleware	Interceptors / Guards

9. Flujo completo

En Node.js

1. Request → Route
2. Route → Controller
3. Controller → Service
4. Service → Data

En Ionic

1. Usuario → Página (UI)
2. Página → Service
3. Service → API/backend
4. Datos → UI

10. Caso práctico ampliado

Modificar el servicio:

```
async getUsers() {  
  const response = await fetch('https://dummyjson.com/users');  
  const data = await response.json();  
  return data.users;  
}
```

11. Conceptos técnicos implicados

- Async/await
- Programación funcional (**filter**)
- Arquitectura por capas
- Manejo de estado
- Event Loop (internamente)
- Separación de responsabilidades

12. Mejora progresiva

Puedes evolucionar este proyecto añadiendo:

- 1. Formulario (crear usuario)**
- 2. Navegación entre páginas**
- 3. Persistencia local**
- 4. Consumo de tu API Node real**
- 5. Interceptors (equivalente a middleware)**
- 6. Gestión de errores global**

13. Buenas prácticas

- No meter lógica en HTML
- Centralizar llamadas API en services
- Usar interfaces para tipado
- Separar responsabilidades
- Evitar duplicación de código

1. Opción básica: navegador (ionic serve)

Uso

ionic serve

Esto abre la app en:

<http://localhost:8100>

Qué estás simulando

- WebView del dispositivo
- Interfaz Ionic completa
- Navegación y componentes

Limitaciones

- No acceso a hardware (cámara, GPS)
- Diferencias con dispositivos reales

Ventajas

- Rápido
- No requiere instalación adicional
- Ideal para desarrollo inicial

2. Simulación con Chrome DevTools (modo móvil)

Una mejora sobre `ionic serve`.

Pasos

1. Ejecuta:

`ionic serve`

2. En Chrome:

- Click derecho → Inspect
- Activar modo dispositivo (icono móvil)

3. Seleccionar dispositivo:

- iPhone
- Pixel
- iPad

Qué permite

- Simular resoluciones móviles
- Testear responsive design
- Simular user agent

Limitaciones

- Sigue siendo navegador
- No simula sensores reales

3. Emulador Android (recomendado)

Requisitos

- Android Studio instalado
- SDK Android
- Emulador configurado

Instalación

1. Descargar Android Studio

<https://developer.android.com/studio>

2. Crear emulador

- Abrir Android Studio
- Device Manager
- Crear AVD (Android Virtual Device)

Integración con Ionic

Añadir Android al proyecto

ionic build

npx cap add android

Abrir en Android Studio

npx cap open android

Ejecutar en emulador

- Pulsar Run dentro de Android Studio

Qué permite

- Simulación realista del sistema Android
- Acceso a:
 - sensores virtuales
 - almacenamiento
 - red

Limitaciones

- Consumo de recursos alto
- Configuración inicial más compleja

4. Simulador iOS (solo macOS)

Dado que estás en Mac, esta es una de las mejores opciones.

Requisitos

- Xcode instalado desde App Store

Uso

`ionic build`

`npx cap add ios`

`npx cap open ios`

Ejecutar

- En Xcode → botón Run
- Se abre iOS Simulator automáticamente

Ventajas

- Muy fiel al comportamiento real
- Excelente rendimiento en Mac
- Integración total con Ionic

Limitaciones

- Solo disponible en macOS
- Requiere Xcode

5. Live Reload con emulador

Permite ver cambios en tiempo real sin recompilar.

Android

```
ionic cap run android -l --external
```

iOS

```
ionic cap run ios -l --external
```

Qué hace

- Lanza servidor local
- Conecta emulador con tu máquina
- Refresca automáticamente

6. Uso híbrido profesional

En entornos reales se usa combinación:

1. Navegador → desarrollo rápido
2. Emulador → testing funcional
3. Dispositivo real → validación final

7. Comparativa técnica

Método	Realismo	Velocidad	Hardware
ionic serve	Bajo	Muy alto	No
Chrome DevTools	Bajo	Alto	No
Android Emulator	Alto	Medio	Sí
iOS Simulator	Muy alto	Alto	Sí

Problemas comunes

Emulador lento

- Activar virtualización (Intel/Apple Silicon OK)
- Usar imágenes x86_64 o ARM adecuadas

Capacitor no sincroniza

`npx cap sync`

Cambios no reflejados

`ionic build`